PRÁCTICA SPRING SECURITY - ONE PIECE

La Marina de One Piece necesita implementar un sistema de seguridad para controlar el acceso a su aplicación de gestión de piratas. El Almirante Akainu ha ordenado que se establezcan diferentes niveles de acceso según el rango militar de cada usuario, de forma similar a como se hizo en la aplicación del Colegio.

Para ello añadiremos Spring Security a nuestro proyecto de One Piece. Puedes usar el que tengo subido a GitHub o el tuyo. Lo añadiremos tanto en Thymeleaf como en un API Rest (te lo doy yo implementado esto ultimo, un pequeño select de cada crud)

ROLES DE LA MARINA

El sistema de la Marina tendrá tres niveles de acceso basados en rangos militares:

- **almirante**: Tendrá acceso completo a todos los recursos de la aplicación (equivalente al rol "director" del colegio).
- **vicealmirante** : Podrá gestionar piratas y tripulaciones. Tendrá acceso a:
  - CRUD de Piratas
  - CRUD de Tripulaciones (incluyendo añadir/eliminar miembros)
- **capitán**: Solo podrá gestionar las recompensas. Tendrá acceso a:
  - CRUD de recompensas

Crea la base de datos de usuario y roles como hicimos en Colegio

```sql
CREATE TABLE users (
    user_id INT NOT NULL AUTO_INCREMENT,
    user_name VARCHAR(255) NOT NULL,
    user_passwd VARCHAR(255) NOT NULL,
    user_email VARCHAR(255) UNIQUE NOT NULL,
     PRIMARY KEY (user_id)
);
CREATE TABLE roles (
    role_id INT NOT NULL AUTO_INCREMENT,
```

```sql
    user_id INTEGER,
    user_role VARCHAR(255),
    FOREIGN KEY (user_id) REFERENCES users(user_id),
    PRIMARY KEY (role_id)
);
```

E inserta los datos:

```sql
INSERT INTO users (user_name, user_passwd, user_email) VALUES
('akainu', '$2a$12$ADsrcfWYwiZnhSE8fvf9ZeoqOUiYAivBNLN4pP6u/p9ZQnb37rsa6',
'akainu@marina.com'),
('smoker', '$2a$12$DnSl16f8ZlSDIf.ZyNCnu.vhmsM7w3.uhL5zvCMfrG/DfKohGhl.W',
'smoker@marina.com'),
('coby', '$2a$12$TKD4jhQjoAGocab.sYGVOOWH8W6meYk3bHMLAgYAhI2SOP6nZRerK',
'coby@marina.com');

INSERT INTO roles (user_id, user_role) VALUES
(1, 'almirante'),
(2, 'vicealmirante'),
(3, 'capitan');
```

La contraseña esta encriptada, como la otra vez es 1234

2. VISTAS THYMELEAF

- Deberás implementar Spring Security en tu aplicación de One Piece siguiendo los mismos pasos que en
  la aplicación del Colegio, pero adaptándolos a tu proyecto:

2.1. Añadir dependencia de Spring Security

- Tendrás que añadir las dependencias necesarias para hacer funcionar Spring Security

2.2. Crear las entidades User y Role

- Como con colegio, crear las entidades User y Role
- Deben tener la relación bidireccional correcta entre User y Role (`@OneToMany` y `@ManyToOne`)

- Los nombres de las columnas deben coincidir con los de la base de datos
- Usa `FetchType.EAGER` para cargar los roles junto con el usuario

2.3. Crear el Repository de Usuario

- Crea la interfaz `UserRepository` que extienda de `CrudRepository` e incluya un método personalizado con `@Query` para buscar un usuario por su nombre de usuario

2.4. Crear el servicio UserDetailsService

- Crea un nuevo paquete llamado `security` y dentro de él implementa la clase `DetallesUsuarioService` que implemente la interfaz `UserDetailsService` de Spring Security
- Debe cargar el usuario desde la base de datos usando el `UserRepository`
- Debe convertir los roles de tu entidad a `GrantedAuthority`
- Debe devolver un objeto `UserDetails` con el username, password y authorities

2.5. Configurar Spring Security

- Crea la clase `SecurityConfig` en el paquete `security` con las anotaciones `@Configuration` y `@EnableWebSecurity` .
- Métodos requeridos:
  - `passwordEncoder()` : Debe retornar un BCryptPasswordEncoder
  - `authenticationProvider()` : Debe configurar el DaoAuthenticationProvider con tu `DetallesUsuarioService` y el `passwordEncoder`
  - `authenticationManager() :` Debe exponer el `AuthenticationManager``
  - `securityFilterChain()` : Configuración principal donde defines la connfiguración de acceso por roles
- Configuración de acceso por roles:
  - URLs públicas (sin autenticación): `/` , `/login` , `/accesoDenegado` , `/css/**`
  - `/home/**` : Cualquier usuario autenticado
  - `/piratas/**` : Solo usuarios con rol almirante o vicealmirante
  - `/recompensas/**` : Solo usuarios con rol almirante o capitán
  - `/tripulaciones/**` : Solo usuarios con rol almirante o vicealmirante

- Cualquier otra URL: Solo rol `almirante`
- Form-based authentication con página de login personalizada en `/login`
- Logout funcional que redirija a `/login?logout`
- Página de acceso denegado en `/accesoDenegado`

2.6. Actualizar el MainController

- Modifica o crea tu `MainController` con las siguientes rutas:
  - `/` → redirige a `/login`
  - `/login` → muestra la vista de login
  - `/home` → muestra la página principal (index), debe recibir `@AuthenticationPrincipal` `UserDetails` para mostrar el usuario y sus roles
  - `/accesoDenegado` → muestra la página de error cuando un usuario intenta acceder a un
    recurso sin permisos

2.7. Vistas HTML

- Añade las vistas de login y accessoDenegado. Si has cogido el proyecto de GitHub ya las tendrás,
  si no añádelas/créalas.
- Modifica tu página principal (`index.html`) para:
  - Mostrar el nombre del usuario logueado usando `th:text="${usuario}"`
  - Mostrar el rol del usuario usando `th:text="${rol}"`
  - Incluir un botón/enlace de logout que haga un POST a `/logout`

2.8. Probar la aplicación

- Para probar que todo ha ido bien comprueba que (es lo que haré cuando corrija tu practica) :
  1. Login funcional: Los tres usuarios pueden iniciar sesión con sus credenciales
  2. Restricciones por rol:
     - Usuario `almirante` puede acceder a TODO
     - Usuario `vicealmirante` puede acceder solo a Piratas y Tripulaciones

    - Usuario `capitan` puede acceder solo a recompensas

3. Acceso denegado: Cuando un usuario intenta acceder a un recurso sin permisos, se muestra
   la página `accesoDenegado.html`
4. Logout funcional: El botón de cerrar sesión funciona correctamente y redirige al login
5. Información del usuario: En la página principal se muestra correctamente el nombre y rol
   del usuario logueado

3. IMPLEMENTACIÓN DE API REST CON JWT

- Vamos a implementar también la seguridad en la API Rest (te la doy yo en github, no tienes que
  implementar esa parte)

3.1. Añadir dependencias JWT

- Añade de las dependencias en tu `pom.xml` para trabajar con JWT. Mismas que con colegio:

```xml
<dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt-api</artifactId>
    <version>0.12.5</version>
</dependency>

<dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt-impl</artifactId>
    <version>0.12.5</version>
    <scope>runtime</scope>
</dependency>

<dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt-jackson</artifactId>
    <version>0.12.5</version>
```

```
    <scope>runtime</scope>
</dependency>
```

3.2. Configurar JWT en application.properties

- Añade estas propiedades en tu archivo `application.properties` :

```properties
jwt.secret=marinaOnePieceSecretKeyMuyLargaYCompleja123456789
jwt.expiration=86400000
```

  - Puedes poner los valores que quieras

3.3. Crear los DTOs

- Crea los DTO de login/user y JwtDTO: Uno para recibir las credenciales. Otro para enviar la
  respuesta con el token

3.4. Crear JwtTokenProvider

- Crea un nuevo paquete `security.jwt` y dentro de él la clase `JwtTokenProvider.java` .
- Esta clase debe tener tres métodos principales:
  - `generateToken(Authentication authentication)` : Genera un JWT cuando el usuario se
    autentica
  - `getUsernameFromToken(String token)` : Extrae el username del JWT
  - `validateToken(String token)` : Valida si un JWT es correcto y no ha expirado

3.5. Crear JwtAuthenticationFilter

- En el mismo paquete `security.jwt` , crea la clase `JwtAuthenticationFilter.java` que extienda de
  `OncePerRequestFilter` .
- Esta clase interceptará TODAS las peticiones HTTP a la API y:
  1. Extraerá el JWT del header `Authorization: Bearer <token>`
  2. Validará el token usando `JwtTokenProvider`

3. Si es válido, autenticará al usuario en Spring Security
4. Si no es válido o no existe, dejará pasar la petición sin autenticar

3.6. Actualizar SecurityConfig con segunda cadena de filtros

- Modifica tu `SecurityConfig.java` para añadir un segundo `SecurityFilterChain` específico para la API.
  - Usar `@Order(1)` en el SecurityFilterChain existente de Thymeleaf
  - Crear un nuevo método con `@Order(2)` para la API que:
    - Use `securityMatcher("/v1/**")` para aplicarse solo a rutas de API
    - Configure `sessionManagement` con `SessionCreationPolicy.STATELESS`
    - Define permisos similares a Thymeleaf pero para rutas `/v1/` :
      - `/v1/login` → público (permitAll)
      - `/v1/piratas/**` y `/v1/recompensas/**` → almirante y vicealmirante
      - `/v1/tripulaciones/**` → almirante y vicealmirante
      - Resto → authenticated
    - Configura el controlador de errores con `authenticationEntryPoint` y `accessDeniedHandler` que
      devuelvan JSON
    - Añade el filtro JWT con `addFilterBefore(jwtAuthenticationFilter, UsernamePasswordAuthenticationFilter.class)`

3.7. Crear AuthController

- Crea un nuevo paquete controladores.rest y dentro de él la clase AuthController.java .
- Debe tener un endpoint POST `/v1/login` que:
  1. Reciba un `LoginDTO` en el body
  2. Autentique al usuario con `AuthenticationManager`
  3. Genere un JWT usando `JwtTokenProvider`
  4. Devuelva un `JwtDTO` con el token y los roles del usuario

3.8. Probar la API con Bruno o Postman

- Para probar que todo ha ido bien comprueba que (es lo que haré cuando corrija tu practica)

:

1. Login exitoso: Hacer POST a `/v1/login` con credenciales válidas debe devolver un token
   JWT
2. Acceso sin token: Intentar acceder a cualquier endpoint protegido sin token debe devolver
   error 401 (Unauthorized)
3. Acceso con token: Incluir el token en el header `Authorization: Bearer [token]` debe
   permitir el acceso
4. Restricciones por rol:
   - Capitán debe poder acceder a `/v1/recompensas`
   - Vicealmirante debe poder acceder solo a `/v1/tripulaciones` y `/v1/piratas`
   - Capitan intentando acceder a `/v1/tripulaciones`os debe recibir error 403 (Forbidden)

EVALUACIÓN

- Practica asociada a: RA4
- Plagio: El plagio supondrá un cero tanto al plagiador como al plagiado. Se considerará plagio el
  hecho de que el 25% o más del código sea idéntico.
- Penalizaciones:
  - -0.5 pts: Variables no descriptivas o no en camelCasel
  - -1 pt: Estructura de paquetes incorrecta o desorganizada

PARTE 1: SPRING SECURITY CON THYMELEAF (5 puntos)

CRITERIO	EXCELENTE	ACEPTABLE	INSUFICIENTE
Implementación completa - 2 pts	Todas las clases necesarias creadas y configuradas correctamente. Código **adaptado a One Piece** (nombres de clases, variables, comentarios cambiados del ejemplo del colegio). Estructura de paquetes correcta.	Implementación completa pero código es **copia casi exacta** del colegio con solo cambios mínimos. O bien adaptado pero falta alguna clase/configuración menor. **(1 pt)**	Falta implementación de partes importantes (SecurityConfig incompleto, sin UserDetailsService, etc.) o código que no compila. **(0 pts)**
Funcionamiento - Login y Logout - 1.5 pts	Los **3 usuarios** (akainu, smoker, coby) pueden hacer login con "1234". Logout funciona y redirige a login. Usuario y rol se muestran correctamente en home.	Funciona pero **1 usuario tiene problemas** al hacer login, o logout no redirige bien, o usuario/rol no se muestran correctamente. **(0.75 pts)**	Login no funciona para **2+ usuarios**, o logout no funciona, o múltiples errores. **(0 pts)**
Funcionamiento - Restricciones por Rol - 1.5 pts	**Los 3 roles funcionan perfectamente**: Almirante accede a TODO. Vicealmirante accede a piratas/tripulaciones (NO recompensas). Capitán accede solo a recompensas (NO piratas/tripulaciones). Página accesoDenegado se muestra.	**1 rol tiene permisos incorrectos** (ej: vicealmirante puede acceder a recompensas) o accesoDenegado no se muestra en algún caso. **(0.75 pts)**	**2+ roles con permisos incorrectos** o todos pueden acceder a todo (sin restricciones). **(0 pts)**

PARTE 2: API REST CON JWT (5 puntos)

CRITERIO	EXCELENTE	ACEPTABLE	INSUFICIENTE
Implementación completa JWT - 2 pts	Dependencias JWT añadidas. Properties configuradas. DTOs, JwtTokenProvider, JwtAuthenticationFilter, SecurityConfig con @Order(2) y AuthController implementados. Código **adaptado a One Piece** (nombres, comentarios).	Implementación completa pero código es **copia casi exacta** del colegio con cambios mínimos. O bien adaptado pero falta algún componente menor (DTOs sin todos los campos, etc.). **(1 pt)**	Falta implementación importante (sin JwtTokenProvider, sin segundo SecurityFilterChain, sin AuthController) o no compila. **(0 pts)**
Funcionamiento - Autenticación JWT - 1.5 pts	POST /v1/login devuelve **token JWT válido** con estructura correcta (token, usuario, roles). Endpoints protegidos dan **error 401** sin token. Con token en header "Authorization: Bearer [token]" **permite el acceso**.	Funciona pero **un aspecto falla**: token se genera pero sin roles, o 401 no es JSON correcto, o token válido a veces no autentica. **(0.75 pts)**	Login no genera token, o token no permite acceso, o **2+ aspectos fallan**. **(0 pts)**
Funcionamiento - Restricciones API - 1.5 pts	**Los 3 roles funcionan perfectamente en API**: Almirante accede a todos los endpoints. Vicealmirante accede a /v1/piratas y /v1/tripulaciones (**403 en recompensas**). Capitán accede solo a /v1/recompensas (**403 en otros**). Errores devuelven JSON.	**Un rol tiene permisos incorrectos** en API (ej: capitan puede acceder a tripulaciones) o errores 403/401 no devuelven JSON correcto. **(0.75 pts)**	**2+ roles con permisos incorrectos** o todos pueden acceder a todo sin restricciones. **(0 pts)**